MNMKernel: Design and Implementation of a Baremetal 32-bit x86 Operating System Kernel with Demand Paging, ELF Userland Execution, and Preemptive Multitasking

Swati Yelamanchili
swati.yelamanchili@gmail.com

Abhiroop Pullepu
09abhiroop@gmail.com

***Abstract*—This paper presents MNMKernel, a fully functional 32-bit x86 operating system kernel written from scratch in C and x86 Assembly. The kernel implements the complete foundational subsystems required of a modern OS: a multiboot-compliant bootloader, a Global Descriptor Table (GDT) and Interrupt Descriptor Table (IDT) for hardware abstraction, a bitmap-based Physical Memory Manager (PMM), a two-level x86 page directory/table MMU with hardware demand paging, a Linux-compatible `int $0x80` syscall interface, an ELF32 binary loader capable of executing native static Linux binaries in Ring 3 userland, and a Programmable Interval Timer (PIT)-driven preemptive round-robin task scheduler with full context switching. The system boots and executes in QEMU. All components were implemented without any standard library or external OS dependency. This paper documents the architectural decisions, implementation details, and observed behaviors of each subsystem, with emphasis on the MMU demand paging mechanism and the Ring 0/3 privilege transition.**

***Index Terms*—operating systems, x86 kernel, demand paging, ELF loader, preemptive multitasking, syscall ABI, privilege rings, baremetal**

I. Introduction

Building an operating system kernel from first principles remains one of the most demanding exercises in computer science education and systems research. Unlike application-level programming, kernel development requires a precise understanding of hardware behavior: how the CPU enforces privilege separation, how the Memory Management Unit (MMU) translates virtual to physical addresses, and how interrupt delivery transforms the execution model from sequential to event-driven.

MNMKernel is a complete 32-bit x86 baremetal operating system implemented from scratch. It makes no use of libc, POSIX libraries, or any existing kernel code. Every subsystem — from the multiboot header in assembly to the ELF program segment loader in C — was written and debugged directly against the x86 hardware specification.

The system was designed with the following goals:

- Boot on real x86 hardware (or QEMU) via a Multiboot-compliant bootloader
- Establish full privilege separation between kernel (Ring 0) and user processes (Ring 3)
- Implement hardware-level virtual memory with transparent demand paging
- Execute native Linux 32-bit ELF binaries without modification
- Support preemptive multitasking via real hardware timer interrupts
- Provide a Linux-compatible syscall boundary for userland I/O

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture. Sections IV through X detail each subsystem. Section XI presents results and observed behavior. Section XII discusses limitations and future work. Section XIII concludes.

II. Related Work

Educational OS kernels have a rich lineage. MINIX [1] demonstrated that a microkernel architecture could be used as a teaching tool. xv6 [3] from MIT provides a clean UNIX-like kernel for pedagogical use. OSDev community resources [4] document baremetal x86 bringup conventions. MNMKernel differs from these by targeting the complete end-to-end pipeline — from raw multiboot boot to executing unmodified standard ELF binaries from a ramdisk VFS — in a single, self-contained codebase without relying on pre-existing kernel skeletons.

III. System Architecture

MNMKernel is organized as a monolithic kernel with clearly separated subsystems. The boot sequence proceeds as follows:

1) **Boot:** The GRUB/QEMU bootloader loads `kernel.bin` and passes control to `_start` in `boot.S`.
2) **CPU Init:** `kernel_main()` initializes the PMM, GDT, IDT, and hardware paging.
3) **InitRD:** The multiboot module list is parsed to locate the ramdisk.
4) **ELF Loading:** Named ELF binaries are read from the ramdisk and loaded into virtual memory.

5) **Scheduling:** Tasks are registered with the scheduler; the PIT timer is armed.
6) **Execution:** The `hlt` loop yields to timer IRQs, which trigger context switches into Ring 3 userland.

A. Memory Map

The kernel is linked at physical address `0x100000` (1 MB), per the multiboot convention. The first 4 MB of physical memory is identity-mapped and reserved for the kernel, VGA buffer (`0xB8000`), and BIOS structures. User processes are loaded at virtual address `0x80000000` and above.

IV. Bootloader and CPU Initialization

A. Multiboot Entry Point

The entry point `boot.S` embeds a Multiboot header in a dedicated `.multiboot` section:

```
.set MAGIC,    0x1BADB002
.set FLAGS,    ALIGN | MEMINFO
.set CHECKSUM, -(MAGIC + FLAGS)
.long MAGIC
.long FLAGS
.long CHECKSUM
```

The bootloader verifies the checksum and passes the magic value and a pointer to the `multiboot_info` structure in registers `eax` and `ebx`. The entry stub sets the stack pointer to a statically allocated 16 KB BSS region and calls `kernel_main()`.

B. Linker Script

The linker script places the kernel at the 1 MB physical boundary, aligning each section (`.text`, `.rodata`, `.data`, `.bss`) on 4 KB boundaries — matching the page granularity of the MMU. This allows the identity-mapping code to protect the kernel by simply marking the first 1024 page table entries as present.

V. Descriptor Tables: GDT and IDT

A. Global Descriptor Table

The GDT contains six entries:

TABLE I
GDT Layout

Index	Segment	Access	DPL
0	Null	–	–
1	Kernel Code	0x9A	0
2	Kernel Data	0x92	0
3	User Code	0xFA	3
4	User Data	0xF2	3
5	TSS	0x89	0

Selector `0x1B` (index 3, RPL=3) is used as CS for Ring 3. Selector `0x23` (index 4, RPL=3) is used for DS/SS in userland. The GDT is loaded via the `lgdt` instruction and segment registers are reloaded with a far jump.

B. Task State Segment

The TSS is critical for Ring 3 → Ring 0 transitions. When a hardware interrupt fires while the CPU is in Ring 3, it uses `TSS.esp0` and `TSS.ss0` to switch to the kernel stack. MNMKernel updates `esp0` on every context switch to point to the current task's kernel stack, preventing stack corruption across tasks.

C. Interrupt Descriptor Table

The IDT contains 256 gate entries. MNMKernel populates entries for: CPU exceptions (0, 1), the page fault handler (14), the PIT IRQ (32), the keyboard IRQ (33), and the syscall gate (128, `int $0x80`). All ISR stubs are generated from two macros in `isr.S`:

```
.macro ISR_NOERRCODE num
  isr\num:
    cli
    push $0        ; dummy error code
    push $\num
    jmp isr_common_stub
.endm

.macro ISR_ERRCODE num
  isr\num:
    cli
    push $\num     ; CPU already pushed error code
    jmp isr_common_stub
.endm
```

The common stub saves all general-purpose registers with `pusha`, switches segment registers to the kernel data segment, calls the C handler, then restores registers and returns with `iret`.

VI. Memory Management

A. Physical Memory Manager

The PMM uses a flat bitmap where each bit represents one 4 KB physical frame. The bitmap covers 1024 32-bit words, managing up to 32,768 frames (128 MB of RAM):

```
#define PMM_BLOCKS 1024
static uint32_t pmm_bitmap[PMM_BLOCKS];
```

During initialization, the first 32 words are set to `0xFFFFFFFF`, marking the first 4 MB (1024 frames) as reserved — protecting the kernel, VGA buffer, and page structures from being reallocated. `pmm_alloc_frame()` performs a linear scan for the first clear bit, marks it, and returns the corresponding physical address as `(i*32 + j) * 4096`.

B. Two-Level Hardware Paging

MNMKernel implements the standard x86 two-level page table structure. A single 4 KB-aligned page directory of 1024 32-bit entries is maintained. Each entry, if present, points to a page table of 1024 entries, each mapping a 4 KB physical frame.

Initialization:

1) All 1024 directory entries are marked not-present (`0x02`: R/W, supervisor).

2) A first page table identity-maps addresses `0x00000000–0x003FFFFF`, covering the kernel and VGA memory, with flags `0x03` (present, R/W).
3) The first directory entry is set to point to this page table.
4) The page directory physical address is loaded into CR3.
5) Bit 31 of CR0 is set to enable paging.

C. Demand Paging

All additional memory allocation is deferred until the moment of access. When the ELF loader copies segments to addresses above `0x80000000`, those addresses are unmapped, triggering hardware Page Fault exception #14. The page fault handler:

1) Reads the faulting virtual address from CR2.
2) Computes the directory index (`VA >> 22`) and table index (`(VA >> 12) & 0x3FF`).
3) If the directory entry is not present, allocates a new page table from a static pool and installs it with user/R/W/present flags (`0x07`).
4) Calls `pmm_alloc_frame()` to obtain a physical frame.
5) Maps the frame in the page table with flags `0x07`.
6) Invalidates the TLB entry via `invlpg`.
7) Zero-initializes the newly mapped page.
8) Returns, allowing the faulting instruction to transparently retry.

This design means the ELF loader requires no pre-allocation logic — writing to an unmapped page is indistinguishable from writing to a mapped one from the loader's perspective.

VII. ELF32 Binary Loader

MNMKernel implements a complete ELF32 loader. Given a pointer to ELF binary data (from the InitRD), it:

1) Validates the magic bytes `0x7F 'E' 'L' 'F'`.
2) Reads the ELF header to locate the program header table at offset `e_phoff`.
3) Iterates over `e_phnum` program headers; for each `PT_LOAD` segment (type 1):
   - Copies `p_filesz` bytes from the ELF data to virtual address `p_vaddr`.
   - Zero-initializes the remaining `p_memsz − p_filesz` bytes (BSS).
4) Returns `e_entry` as the process entry point.

The copy loop writes directly to userland virtual addresses. Each write that hits an unmapped page triggers the demand pager, which allocates and maps a frame before the write completes. This coupling between the ELF loader and the page fault handler eliminates any need for upfront virtual memory reservation.

VIII. Ring 3 Userland and Syscall ABI

A. Ring 3 Entry via IRET

The initial transition from Ring 0 to Ring 3 is performed by `switch_to_user_mode()`, which constructs a synthetic interrupt return frame on the kernel stack:

```
pushl $0x23     ; SS  (User Data, RPL=3)
pushl %0        ; ESP (user stack top)
pushf           ; EFLAGS
orl $0x200, (%esp)  ; Enable IF
pushl $0x1B     ; CS  (User Code, RPL=3)
pushl %1        ; EIP (entry point)
iret
```

The `iret` instruction atomically loads CS:EIP, SS:ESP, and EFLAGS from the stack, dropping the CPU into Ring 3 at the process entry point.

B. Syscall Interface

User processes invoke kernel services via `int $0x80` with the syscall number in `eax` and arguments in `ebx`, `ecx`, `edx`. Table II lists the implemented syscalls.

TABLE II
Implemented Syscalls

eax	Name	Behavior
1	sys_exit	Halt kernel (`cli; hlt`)
3	sys_read	Read from open InitRD file
4	sys_write	Write to serial (fd=1 only)
5	sys_open	Open file by name from InitRD
6	sys_close	Close open file descriptor
*	(unimplemented)	Return `-ENOSYS` (-38)

The syscall handler executes in Ring 0, directly within the interrupt frame saved by `isr_common_stub`. Return values are written back to `regs->eax` before `iret` restores the frame to Ring 3.

IX. Preemptive Multitasking

A. PIT Timer

The Intel 8253/8254 Programmable Interval Timer is configured in mode 3 (square wave, repeating) via port `0x43`. The frequency divisor is computed from the base clock of 1,193,180 Hz:

```
uint32_t divisor = 1193180 / frequency;
outb(0x43, 0x36);
outb(0x40, divisor & 0xFF);
outb(0x40, (divisor >> 8) & 0xFF);
```

MNMKernel initializes the PIT at 5 Hz for debugging visibility. Each PIT tick fires IRQ0 (mapped to interrupt vector 32).

B. Task Control Block

Each task is represented by a `tcb_t` containing its saved kernel stack pointer (`esp`), dedicated kernel stack top address, CR3 value for address space switching, and a state field (unused=0, running=1, ready=2).

C. Context Switch Mechanism

The IRQ common stub differs critically from the ISR stub. After calling `irq_handler()`, it uses the *return value* of that function as the new ESP:

```
push %esp
call irq_handler
mov %eax, %esp     ; switch to new task's kernel
    stack
pop %eax           ; restore DS from new stack
...
iret               ; return to new task's Ring 3
```

The `schedule()` function, called from `irq_handler` on IRQ0:

1) Saves the current `esp` into the active TCB.
2) Advances to the next ready task (round-robin).
3) Loads the new task's CR3, flushing the TLB.
4) Updates `TSS.esp0` to the new task's kernel stack.
5) Returns the new task's saved `esp`.

When `irq_common_stub` executes `iret` using the new task's stack pointer, it resumes execution of the switched-to task exactly where it was interrupted.

X. Initial Ramdisk and Virtual Filesystem

MNMKernel defines a simple ramdisk archive format. The first 4 bytes encode the file count. Immediately following is an array of `initrd_file_header_t` structures, each containing a filename string, a byte offset within the archive, and a length. Raw file data follows the header array.

The `tools/make_initrd` utility packs files into this format. QEMU loads the resulting `initrd.img` as a multiboot module; the kernel extracts its address from the `multiboot_mod_list` and calls `init_initrd()`.

File lookups are performed by linear name comparison. The `open`/`read`/`close` syscall sequence allows Ring 3 processes to read files from the ramdisk as though from a real filesystem.

XI. Results and Behavior

A. Boot Sequence

When launched with:

```
qemu-system-i386 -kernel kernel.bin \
  -initrd initrd.img -m 128M -nographic
```

The kernel produces serial output walking through all initialization phases. GDT, IDT, PMM, and paging initialization complete within the first millisecond of execution. The initrd is parsed and its file table is printed.

B. Demand Paging in Action

Loading the `cat` ELF binary produces the following trace on serial output:

```
[ELF Loader] Found valid ELF file. Entry: 0x08048054
[ELF Loader] PT_LOAD segment: VAddr: 0x08048000
[MMU] Page Fault Trapped! VA: 0x08048000
[MMU] Granted physical frame 0x00500000
[MMU] Transparently Returning to application.
```

Each page fault is handled in under one timer tick, and the loader proceeds transparently. A total of N page faults are triggered per `PT_LOAD` segment of N pages.

C. Preemptive Scheduling

With the PIT running at 5 Hz, IRQ0 fires every 200ms. The `schedule()` function correctly rotates between tasks. Serial output from multiple tasks is interleaved, confirming round-robin preemption.

D. Syscall Execution

The `cat` Ring 3 binary successfully executes the following syscall sequence:

1) `sys_open("hello.txt")` → returns fd 3
2) `sys_read(3, buf, 64)` → reads from InitRD
3) `sys_write(1, buf, n)` → emits to serial
4) `sys_close(3)` → clears state
5) `sys_exit(0)` → halts

XII. Limitations and Future Work

The current implementation has several known limitations:

- **Shared address space:** All tasks share the same page directory (CR3). Per-process isolation requires allocating a separate page directory per task — this is the next planned feature.
- **Static page table pool:** The kernel maintains a fixed pool of 16 dynamic page tables. A production system would allocate page tables from the PMM directly.
- **Single open file:** The VFS maintains one global open-file state, limiting concurrent file access.
- **No keyboard driver:** IRQ1 is wired but unimplemented; interactive shell support is planned.
- **No pmm_free integration:** `pmm_free_frame()` exists but is not called on task exit, causing a physical memory leak.
- **sys_exit halts:** Rather than terminating only the calling task, `sys_exit` halts the entire CPU. Task teardown and scheduler removal are future work.

Future extensions include: per-process address spaces with fork/exec, a keyboard-driven interactive shell, a pipe IPC mechanism, and ext2 filesystem support on a virtual disk.

XIII. Conclusion

MNMKernel demonstrates that a functional 32-bit x86 operating system — with hardware paging, ELF userland execution, a Linux-compatible syscall ABI, and preemptive multitasking — can be built from scratch as a single-developer project in C and x86 Assembly. Each subsystem was implemented directly against the x86 hardware specification without any OS library support. The demand paging mechanism, in particular, provides an elegant coupling between the ELF loader and the MMU that eliminates upfront memory reservation. The project provides a concrete foundation for studying OS internals and serves as a base for future extension toward a more complete microkernel or UNIX-like system.

REFERENCES

[1] A. S. Tanenbaum, *Operating Systems: Design and Implementation*. Englewood Cliffs, NJ: Prentice-Hall, 1987.

[2] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer's Manual*, Volume 3A: System Programming Guide. Intel, 2023.

[3] R. Cox, F. Kaashoek, and R. Morris, "xv6: A Simple, Unix-like Teaching Operating System," MIT CSAIL, 2011. [Online]. Available: https://pdos.csail.mit.edu/6.828/2012/xv6.html

[4] OSDev Wiki, "Bare Bones," [Online]. Available: https://wiki.osdev.org/Bare_Bones

[5] Free Software Foundation, *Multiboot Specification*, version 0.6.96, 2010. [Online]. Available: https://www.gnu.org/software/grub/manual/multiboot/multiboot.html

[6] Tool Interface Standard (TIS) Committee, *Executable and Linkable Format (ELF)*, version 1.2, 1995.

[7] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2005.